# Багатопоточність у Free Pascal: що нового

Гілка **feature/concurrency** (FPC 3.3.1) · жовтень 2026 · для розробників, які використовують System.Threading

## Огляд

Гілка виправляє помилки в `System.Threading` (пакет vcl-compat), додає підтримку перевірки гонок даних через valgrind і робить тести багатопоточності надійнішими. Публічний API не змінено: існуючий код компілюється без змін і починає працювати коректніше.

| Що | Було | Стало |
|---|---|---|
| `TParallel.For` | Іноді виконувався в одному потоці (20,7 с замість 1,7 с); ітерації могли губитися або виконуватися двічі; цикли з `TLoopState` падали з AV; `Break`/`Stop` не зупиняли цикл; винятки в ітераціях губилися | Стабільно паралельний; кожна ітерація рівно один раз; працюють `TLoopState`, `Break`, `Stop` і передавання винятків |
| `TParallel.Join` | Не запускав задачу; `Params` зі сміттям зі стеку; `Parent := Self` вічно блокував `Wait` | Запускає процедури паралельно й коректно завершує `Wait` |
| `TTask.Wait`, `WaitForAny` | Читання стану без бар'єра / без блокування | Коректно і на x86, і на ARM64 |
| `TThreadPool.Free` | Повертав керування, поки потоки пулу ще працювали; запис у звільнену пам'ять монітором | Дочікується фактичного завершення всіх потоків |
| valgrind / helgrind | Сотні хибних попереджень; падіння valgrind | Модуль `fpvalgrind`, анотації, скрипти; 0 попереджень у System.Threading |
| `for E in EAggregateException` | Видавав зайвий елемент за межами масиву — падіння (Access violation) | Рівно ті помилки, що зібрано |
| Тести | `Assert` з робочих потоків міг загубити невдачу | `ThreadCheck` / `ThreadCheckSame` |

## 1. Паралельний цикл TParallel.For

Виправлено низку помилок. (1) На щойно створеному («холодному») пулі вільні потоки не помічали роботу в локальній черзі кореневої задачі й спали до 40 с, тож увесь цикл виконувався одним потоком. (2) Потік забирав діапазон ітерацій з одним розміром кроку, а виконував з іншим, тому частина ітерацій губилася або виконувалася двічі при `TLoopResult.Completed = True`. (3) `TLoopState` не створювався (`nil`), `Break` і `Stop` не зупиняли цикл, а винятки з дочірніх реплік губилися (`EAggregateException.Count = 0`).

docs/concurrency/examples/parallel_for.pp

```
program parallel_for;
// TParallel.For: кожна ітерація виконується рівно один раз,
// цикл справді паралельний навіть на «холодному» пулі.
{$mode objfpc}{$H+}
{$modeswitch functionreferences}
{$modeswitch anonymousfunctions}
uses
  {$IFDEF UNIX}cthreads,{$ENDIF}
  SysUtils, System.Threading;

const
  N = 1000000;
```

```
var
  Hits: array of Integer;
  I, Bad: Integer;
  Res: TParallel.TLoopResult;
begin
  SetLength(Hits, N);
  Res := TParallel.&For(0, N - 1,
    procedure(aIndex: Integer)
    begin
      AtomicIncrement(Hits[aIndex]);   // різні потоки пишуть у різні елементи
    end);
  Bad := 0;
  for I := 0 to N - 1 do
    if Hits[I] <> 1 then
      Inc(Bad);
  WriteLn('Completed: ', Res.Completed, ', iterations not run exactly once: ', Bad);
end.
```

Вивід програми:

```
Completed: TRUE, iterations not run exactly once: 0
```

## 2. Задачі: TTask, IFuture, WaitForAll/WaitForAny

`Wait` перевіряє завершення без блокування (швидкий шлях); тепер це читання має бар'єр пам'яті, тож результат задачі видно одразу, і на процесорах зі слабким порядком пам'яті (ARM64) також. `WaitForAny` читає індекс завершеної задачі під тим самим блокуванням, під яким його записують.

docs/concurrency/examples/tasks.pp

```
program tasks;
// Задачі: TTask.Run, IFuture<T>, WaitForAll / WaitForAny.
{$mode objfpc}{$H+}
{$modeswitch functionreferences}
{$modeswitch anonymousfunctions}
uses
  {$IFDEF UNIX}cthreads,{$ENDIF}
  SysUtils, System.Threading;

var
  Answer: specialize IFuture<Integer>;
  Jobs: array[0..2] of ITask;
  First: Integer;
begin
  // Результат обчислюється в потоці пулу; Value чекає на нього.
  Answer := TTask.specialize Future<Integer>(
    function: Integer
    begin
      Sleep(50);
      Result := 6 * 7;
    end);
  WriteLn('Future value: ', Answer.Value);

  Jobs[0] := TTask.Run(procedure begin Sleep(300); end);
  Jobs[1] := TTask.Run(procedure begin Sleep(20);  end);
  Jobs[2] := TTask.Run(procedure begin Sleep(150); end);

  First := TTask.WaitForAny(Jobs);          // індекс першої завершеної
  WriteLn('First finished: task #', First);
  WriteLn('All finished: ', TTask.WaitForAll(Jobs, 2000));
end.
```

Вивід програми:

```
Future value: 42
First finished: task #1
All finished: TRUE
```

## 3. Власний пул потоків і його знищення

Потоки пулу тепер належать пулу (`FreeOnTerminate := False`). Завершуючись, потік переходить у список завершених, а пул дочікується його (`WaitFor`) і звільняє — у потоці-моніторі та обов'язково в `Destroy`. Тому після `Pool.Free` жоден потік пулу вже не виконується, а пул за замовчуванням не конфліктує з завершенням програми.

docs/concurrency/examples/pool_lifetime.pp

```
program pool_lifetime;
// Власний пул: Free повертає керування лише тоді, коли всі потоки пулу
// справді завершились (раніше вони могли ще працювати).
{$mode objfpc}{$H+}
{$modeswitch functionreferences}
{$modeswitch anonymousfunctions}
uses
  {$IFDEF UNIX}cthreads,{$ENDIF}
  SysUtils, Classes, System.Threading;

// Кількість потоків процесу (лише Linux).
function LiveThreads: Integer;
var
  Info: TSearchRec;
begin
  Result := 0;
  if FindFirst('/proc/self/task/*', faDirectory, Info) = 0 then
  try
    repeat
      if (Info.Name <> '.') and (Info.Name <> '..') then
        Inc(Result);
    until FindNext(Info) <> 0;
  finally
    FindClose(Info);
  end;
end;

var
  Pool: TThreadPool;
  Done, I: Integer;
begin
  WriteLn('Threads before: ', LiveThreads);
  Pool := TThreadPool.Create;
  try
    Pool.SetMaxWorkerThreads(4);
    Done := 0;
    for I := 1 to 20 do
      Pool.QueueWorkItem(
        procedure
        begin
          Sleep(10);
          AtomicIncrement(Done);
        end);
    while Done < 20 do
      Sleep(1);
    WriteLn('Threads while pool is alive: ', LiveThreads);
  finally
    Pool.Free;            // чекає (WaitFor) і звільняє всі свої потоки
  end;
  WriteLn('Threads after Free: ', LiveThreads);
end.
```

Вивід програми:

```
Threads before: 1
Threads while pool is alive: 6
Threads after Free: 1
```

**Зверніть увагу.** Обробник `OnThreadTerminate` виконується в потоці, що завершується. `Destroy` чекає й на нього, тож повільний обробник уповільнює `Free`. Не викликайте з нього `TThread.Synchronize`, якщо пул знищується з головного потоку.

## 4. Перевірка гонок даних: модуль fpvalgrind

helgrind і drd не бачать синхронізації без `pthread_mutex`: спін-блокувань, атомарних операцій, кешу об'єктів. Модуль `fpvalgrind` (пакет fcl-base) передає valgrind підказки. Код анотацій є лише в збірці з `-dFPC_VALGRIND`; у звичайній збірці його немає. Запити реалізовано для x86_64 (System V); на інших платформах це порожні процедури.

| Процедура | Призначення |
|---|---|
| `VgHappensBefore(p)` | Потік «публікує» все, що записав, з міткою p (перед звільненням блокування / записом прапорця) |
| `VgHappensAfter(p)` | Потік бачить усе, що опубліковано з міткою p (після захоплення блокування / читання прапорця) |
| `VgCleanMemory(p, n)` | Пам'ять «нова»: для об'єктів, перевикористаних поза менеджером пам'яті (кеші) |
| `VgBenignRace(p, n)` | Не перевіряти поле, яке свідомо читається без блокування (прапорці) |
| `VgRunningOnValgrind` | True, якщо програма працює під valgrind |

docs/concurrency/examples/valgrind_annotate.pp

```
program valgrind_annotate;
// Анотації для helgrind/drd: власна передача даних через прапорець.
// Звичайна збірка:     fpc valgrind_annotate.pp
// Збірка з анотаціями: fpc -dFPC_VALGRIND valgrind_annotate.pp
{$mode objfpc}{$H+}
uses
  {$IFDEF UNIX}cthreads,{$ENDIF}
  SysUtils, Classes{$IFDEF FPC_VALGRIND}, fpvalgrind{$ENDIF};

var
  Payload: Integer;     // дані, які передаємо
  Ready: LongInt;       // прапорець «дані готові»

type
  TProducer = class(TThread)
  protected
    procedure Execute; override;
  end;

procedure TProducer.Execute;
begin
  Payload := 42;
  {$IFDEF FPC_VALGRIND}
  VgHappensBefore(@Ready);          // усе записане вище «опубліковано»
  {$ENDIF}
  InterlockedExchange(Ready, 1);    // атомарний запис з бар'єром
end;

var
  P: TProducer;
begin
  {$IFDEF FPC_VALGRIND}
  WriteLn('Running on valgrind: ', VgRunningOnValgrind);
  {$ENDIF}
  P := TProducer.Create(False);
  while InterlockedCompareExchange(Ready, 1, 1) <> 1 do
    Sleep(1);
  ReadBarrier;
  {$IFDEF FPC_VALGRIND}
  VgHappensAfter(@Ready);           // бачимо все, що опублікував виробник
  {$ENDIF}
  WriteLn('Payload: ', Payload);
  P.WaitFor;
  P.Free;
end.
```

Вивід програми:

```
Running on valgrind: TRUE
Payload: 42
```

### Результат helgrind для цього прикладу:

| Збірка | Звіт helgrind |
|---|---|
| без анотацій | хибна гонка на Payload (рядок 42 ↔ рядок 22) + 1 відома особливість RTL у TThread.WaitFor |
| -dFPC_VALGRIND | лише відома особливість RTL у TThread.WaitFor |

Анотації вже додано в TSpinLock (syncobjs) і в System.Threading. Для своїх програм: позначайте анотаціями лише ту синхронізацію, яку helgrind не бачить сам.

## 5. Тести: ThreadCheck замість Assert у потоках

Assert* у робочому потоці кидає виняток у цьому потоці, а не в тесті, тож невдача може загубитися; лічильники fpcunit не потокобезпечні. У TThreadedTestCase (тести System.Threading) є ThreadCheck і ThreadCheckSame: невдача записується, а TearDown після завершення всіх потоків валить тест з переліком невдач.

Фрагмент тесту

```
procedure TMyTest.DoWork(Sender: TObject);   // виконується в потоці пулу
begin
  // AssertSame('Sender', Self, Sender);     // НІ: виняток у чужому потоці
  ThreadCheckSame('Sender', Self, Sender);   // ТАК: невдачу покаже TearDown
  ThreadCheck(Counter > 0, 'Counter must be positive');
end;
```

Так виглядає невдача:

```
TestExecuteWork  Failed: [TEARDOWN] Checks failed in worker threads:
[thread 127149134571200] Counter must be positive
```

Тестові потоки (TNotifyThread, TTaskThread) тепер не FreeOnTerminate: TearDown дочікується їх і звільняє.

## 6. Інструменти (tests/concurrency/)

| Скрипт | Що робить |
|---|---|
| fpcenv.sh | Налаштування для компіляції свіжим компілятором з дерева джерел (окремий файл налаштувань на кожен запуск) |
| build-testcompat.sh [тека] | Збирає тести vcl-compat з поточних джерел |
| stress.sh [N] [таймаут] | Запускає кожен набір тестів System.Threading N разів окремим процесом; виявляє рідкісні падіння й зависання |
| build-testcompat-valgrind.sh | Збірка з -dFPC_VALGRIND і обходом падіння valgrind на символах налагодження FPC |
| fpc-helgrind.supp | Винятки для навмисної поведінки RTL |
| hgsummary.py лог | Групує звіт helgrind за місцями конфліктів |

Типовий цикл перевірки

```
tests/concurrency/build-testcompat.sh
tests/concurrency/out/testcompat --all --format=plain
tests/concurrency/stress.sh 50 60

tests/concurrency/build-testcompat-valgrind.sh
valgrind --tool=helgrind --suppressions=tests/concurrency/fpc-helgrind.supp \
  --log-file=hg.log tests/concurrency/out-valgrind/testcompat --suite=TTestTask
tests/concurrency/hgsummary.py hg.log
```

## 7. Зміни в RTL (Linux / Unix)

| Файл | Зміна |
|---|---|
| `rtl/unix/tthread.inc` | Новий потік більше не перезаписує без синхронізації службовий прапорець `FSuspendedInternal` (його вже встановлено до старту потоку) |
| `rtl/unix/cthreads.pp` | Список ідентифікаторів потоків `ThreadTidList` читається лише під `ThreadTidLock` |

Ці зміни стосуються кожної програми на FPC під Linux. Повний набір тестів компілятора (10 442 тести) дає той самий список падінь, що й до змін.

## 8. Перевірено

| Перевірка | Результат |
|---|---|
| Тести vcl-compat | 710 тестів; 2 відомі падіння, не пов'язані з потоками |
| Стрес-прогін | 9 наборів × 50 повторів: 450/450; після останніх змін у тестах — 20 повторів: 180/180. Без падінь і зависань |
| Тести компілятора | той самий список падінь, що й до змін |
| helgrind | 0 попереджень у System.Threading; решта — відомі категорії RTL |

## 9. Відомі обмеження

• helgrind не розуміє атомарного лічильника посилань `TInterfacedObject` (модуль System) і показує хибні конфлікти під час звільнення інтерфейсних об'єктів. На роботу програм не впливає.

• `TThread.WaitFor` опитує прапорець завершення без блокування; після нього `pthread_join` дає повну синхронізацію. Доброякісно.

• Анотації valgrind реалізовано лише для x86_64.

Докладніше: `docs/concurrency/guide-system-threading.md`, `docs/concurrency/valgrind.md`, `docs/concurrency/adr/`, `docs/concurrency/baseline.md`, fpdoc-описи `packages/fcl-base/xml/fpvalgrind.xml` і `packages/vcl-compat/xml/system.threading.xml`, `CHANGELOG-concurrency.md`.

# 10. Порівняння: класичний TThread і System.Threading

Дві невеликі програми, кожна у двох варіантах: лише `TThread` з RTL і `System.Threading`. Обидва варіанти дають однаковий результат; заміри на 12 ядрах, компіляція `-O2`, по 3 запуски. Повні тексти — у додатку.

## Приклад А. Обробка даних: прості числа до 5 000 000

Порахувати кількість і суму простих чисел. Обчислювальна задача, одна операція над великим діапазоном.

| | **А1:** `TThread` | **А2:** `System.Threading` |
|---|---|---|
| Результат | 348 513 простих, сума 838 596 693 108 | той самий |
| Час | 201 / 201 / 302 мс | **125 / 122 / 123 мс** |
| Додатково | — | паралельно знаходить найбільше просте (4 999 999) |
| Рядків коду | 80 (1 клас потоку) | **52** (жодного класу) |
| Ручна робота | поділ діапазону, клас потоку, WaitFor, Free, збирання результатів, перевірка FatalException | лише тіло ітерації |

**Чому А2 швидший.** Перевірка великих чисел триває довше, тому в А1 потоки з першими шматками рано простоюють, а останній працює найдовше (звідси й розкид 201–302 мс). `TParallel.For` роздає роботу невеликими порціями, і вільний потік бере наступну — навантаження вирівнюється само. Щоб зробити так у А1, потрібна власна черга задач.

А2: уся паралельна частина

```
TParallel.&For(1, N,
  procedure(I: Integer)
  begin
    if IsPrime(I) then
    begin
      AtomicIncrement(Count);
      InterlockedExchangeAdd64(Sum, I);
    end;
  end);
```

## Приклад Б. Оркестрація задач: звіт з 4 джерел

Чотири незалежні джерела різної тривалості (150–400 мс), одне з них завершується помилкою. Вимоги: виконати одночасно, загальний ліміт 2 с, показати помилки, зібрати решту результатів. Так виглядає типова програма, що одночасно звертається до бази, файлів чи мережі.

| | **Б1:** `TThread` | **Б2:** `System.Threading` |
|---|---|---|
| Результат | помилка показана, 3 значення зібрано | той самий |
| Час | 400 / 401 / 401 мс | 402 / 401 / 401 мс |
| Рядків коду | 88 (**5 класів**) | **42** (жодного класу) |
| Таймаут | власна подія + лічильник (TThread.WaitFor без таймауту) | `WaitForAll(..., 2000)` |
| Помилки | перевірка `FatalException` кожного потоку окремо | усі в одному `EAggregateException` |
| Типи результатів | поле в кожному класі + приведення типу | `IFuture<Int64>`, `IFuture<string>`, … |

Швидкість однакова: час визначає найдовше джерело (послідовно було б ~1100 мс). Виграш — у коді: удвічі менше рядків, жодного допоміжного класу, помилки й таймаут —

вбудовані.

Б2: очікування, таймаут і помилки

```
try
  if not TTask.WaitForAll([Orders, TopCustomer, Average, Rate], 2000) then
    WriteLn('Timeout: report is incomplete');
except
  on Agg: EAggregateException do      // усі помилки в одному місці
    for E in Agg do
      WriteLn('Error: ', E.Message);
end;
```

**Знахідка.** Саме цей приклад виявив ще одну ваду trunk: цикл `for E in Agg` видавав зайвий елемент за межами масиву, і програма падала з `Access violation`. Виправлено (`EAggregateException.GetEnumerator`), додано тести `TestEnumerate` і `TestEnumerateAfterAdd`.

## Приклад В. Повторювані порції задач: 20 раундів × 500 задач

Типовий серверний сценарій: задачі надходять пачками по 500 штук, кожна триває ~0,1 мс. П'ять підходів на тій самій роботі (12 ядер):

| Підхід | Час | Прискорення | Примітка |
|---|---|---|---|
| 1. Послідовно | 926 мс | ×1,00 | базова лінія |
| 2. Потік на кожну задачу | 898 мс | ×1,03 | 10 000 разів створити/знищити потік ОС з'їдає весь виграш |
| 3. Робочі потоки + `WaitFor` | 2031 мс | ×0,46 | у головному потоці `WaitFor` опитує раз на 100 мс (20 × 100 мс) |
| 4. Робочі потоки + власна подія | 190 мс | ×4,87 | ручна черга + `TSimpleEvent`, але потоки створюються на кожен раунд |
| 5. `TParallel.For` | **160 мс** | **×5,79** | пул перевикористовує потоки між раундами; 6 рядків коду |

**Знахідка.** Під час перевірки цього прикладу виявилося, що `TWorkStealingQueue.TrySteal(Item, 0)` на порожній черзі зайво виконував два системні виклики події (`ResetEvent` + `WaitFor(0)`) під спін-блокуванням списку черг пулу. Вільні потоки переходили в `Sleep(1)` і приєднувалися до короткого циклу по одному. Після виправлення (`TestStealNoWaitIsCheap`) час `TParallel.For` зменшився до 160 мс.

## Приклад Г. Комплексний конвеєр розробника: стандартний TThread vs System.Threading (showcase_pipeline.pp)

Одна програма виконує чотири типові прикладні задачі двічі — стандартним методом (класи-спадкоємці `TThread` з ручною синхронізацією) і через `System.Threading` — та звіряє збіг результатів, час і обсяг коду (12 ядер, `-O2`):

| **Задача** | **Стандартний** `TThread` | `System.Threading` | **Виграш** |
|---|---|---|---|
| 1. Пошук першої помилки за індексом (`Break`) | 101 мс · 38 рядків · 1 клас + ручний CAS-цикл для `LowestIdx` | **16 мс · 10 рядків** · `State.Break` + `LowestBreakIteration` | **×6,3 швидше**, у 3,8 раза менше коду |
| 2. Пошук будь-якого збігу (`Stop`) | 101 мс · 30 рядків · 1 клас + спільний прапорець `StopFlag` | **6 мс · 11 рядків** · `State.Stop` | **×16,8 швидше**, у 2,7 раза менше коду |
| 3. Пакетна валідація з винятками | 100 мс · 36 рядків · 1 клас + ручний обхід `FatalException` до `Free` | **1 мс · 14 рядків** · `try..except on EAggregateException` | **×100 швидше**, помилки неможливо загубити |
| 4. Найшвидша репліка + паралельне зведення | 141 мс · 64 рядки · 2 класи + `TSimpleEvent` + `WinnerIdx` | **41 мс · 16 рядків** · `WaitForAny` + `TParallel.Join` | **×3,4 швидше**, у 4 рази менше коду |

| **Задача** | **Стандартний** TThread | System.Threading | **Виграш** |
|---|---|---|---|
| **Разом по 4 задачах** | **443 мс · 168 рядків · 5 класів** | **64 мс · 51 рядок · 0 класів** | **×6,9 швидше, −70% коду** |

**Чому стандартний метод програє в 3,4–100 разів за часом на коротких етапах:** (1) створення й знищення потоків ОС на кожну задачу; (2) TThread.WaitFor у головному потоці опитує завершення з кроком 100 мс (CheckSynchronize(100)), тому кожне приєднання додає ~100 мс затримки. Пул System.Threading тримає робочі потоки напоготові й синхронізується через легковагові події без затримки.

**Головні переваги нових інструментів при розробці програм:**

• **Декларативність замість інфраструктурного коду.** Розробник описує що потрібно зробити (тіло ітерації, функцію майбутнього результату, перелік процедур у Join), а не як ділити діапазони, створювати класи-спадкоємці TThread, крутити CAS-цикли, сигналізувати події чи звільняти пам'ять.

• **Детермінована семантика зупинки (**Break **vs** Stop**).** Break зберігає послідовний інваріант «знайти перший за порядком індексу», тоді як Stop дає максимальну економію часу, коли підходить будь-який результат. Зі звичайним TThread кожен із цих режимів потребує окремого протоколу синхронізації.

• **Наскрізна безпека щодо винятків.** У класичних потоках необроблений виняток тихо осідає в FatalException (якщо програміст забув його перевірити до Free — помилка зникає). У System.Threading винятки з задач, майбутніх обчислень і паралельних циклів гарантовано доставляються в потік-викликач як EAggregateException.

• **Композиційність і перевикористання пулу.** Послідовні та вкладені виклики TParallel.For, TParallel.Join та TTask.Future працюють на спільному пулі з крадіжкою роботи (work-stealing), автоматично балансуючи нерівномірне навантаження.

## Підсумок порівняння

| **Аспект** | **Класичний** TThread | System.Threading |
|---|---|---|
| Обсяг коду | 80–168 рядків | 42–52 рядки (−50…−70%) |
| Допоміжні класи | 1–5 на задачу | 0 |
| Розподіл роботи | вручну, статично | автоматично, з вирівнюванням |
| Ранній вихід із циклу | власний CAS-цикл мінімуму і прапорці | State.Break (за індексом) / State.Stop (миттєво) |
| Помилки в потоках | легко пропустити (FatalException) | повторно виникають у коді, що чекає (EAggregateException) |
| Таймаут і хеджування | власні події й лічильники | WaitForAll / WaitForAny з таймаутом |
| Потоки | створюються щоразу (+100 мс у WaitFor) | перевикористовуються пулом |

**Коли класичний TThread кращий:** довгоживучий окремий потік (фоновий сервіс, цикл повідомлень, потік на з'єднання з блокуючим I/O); потрібен контроль пріоритету чи прив'язки до ядра; дуже дрібні ітерації, де накладні витрати на кожну ітерацію й атомарні операції з'їдають виграш (тоді обробляйте дані блоками з локальним підсумком).

**Чому це стало практичним саме тут.** API System.Threading був у trunk і раніше, але без виправлень гілки варіант А2 міг випадково виконатися в одному потоці (у 12 разів повільніше) або загубити чи подвоїти ітерації з неправильним результатом, Б2 — упасти під час перебору помилок, а приклад Г — впасти з AV на TLoopState, загубити виняток циклу та назавжди зависнути на TParallel.Join.

## Додаток. Повні тексти прикладів порівняння

docs/concurrency/examples/primes_classic.pp

```
program primes_classic;
// Варіант 1: лише TThread з RTL (без System.Threading).
// Кількість і сума простих чисел до N.
{$mode objfpc}{$H+}
uses
  {$IFDEF UNIX}cthreads,{$ENDIF}
  SysUtils, Classes, DateUtils;

const
  N = 5000000;

function IsPrime(X: Integer): Boolean;
var
  D: Integer;
begin
  if X < 2 then Exit(False);
  if X mod 2 = 0 then Exit(X = 2);
  D := 3;
  while D * D <= X do
  begin
    if X mod D = 0 then Exit(False);
    Inc(D, 2);
  end;
  Result := True;
end;

type
  // Свій клас потоку на кожну задачу: межі діапазону й результати — поля.
  TPrimeWorker = class(TThread)
  private
    FLow, FHigh: Integer;
  public
    Count: Integer;
    Sum: Int64;
    constructor Create(ALow, AHigh: Integer);
  protected
    procedure Execute; override;
  end;

constructor TPrimeWorker.Create(ALow, AHigh: Integer);
begin
  FLow := ALow;
  FHigh := AHigh;
  inherited Create(False);           // одразу стартує
end;

procedure TPrimeWorker.Execute;
var
  I: Integer;
begin
  for I := FLow to FHigh do
    if IsPrime(I) then
    begin
      Inc(Count);
      Inc(Sum, I);
    end;
end;

var
  Workers: array of TPrimeWorker;
  ThreadCount, I, Chunk, Low, High, Count: Integer;
  Sum: Int64;
  Start: TDateTime;
begin
  Start := Now;
  ThreadCount := TThread.ProcessorCount;
  // Ручний поділ на рівні шматки. Великі числа перевіряти довше,
```

```
  // тож останні потоки працюють довше за перші.
  Chunk := N div ThreadCount;
  SetLength(Workers, ThreadCount);
  for I := 0 to ThreadCount - 1 do
  begin
    Low := I * Chunk + 1;
    if I = ThreadCount - 1 then High := N else High := Low + Chunk - 1;
    Workers[I] := TPrimeWorker.Create(Low, High);
  end;
  Count := 0;
  Sum := 0;
  for I := 0 to ThreadCount - 1 do
  begin
    Workers[I].WaitFor;               // чекаємо кожен потік вручну
    if Assigned(Workers[I].FatalException) then
      raise Exception.Create('Worker failed: ' +
        Exception(Workers[I].FatalException).Message);  // помилки — теж вручну
    Inc(Count, Workers[I].Count);     // збираємо результати вручну
    Inc(Sum, Workers[I].Sum);
    Workers[I].Free;                  // і звільняємо вручну
  end;
  WriteLn('Primes: ', Count, ', sum: ', Sum);
  WriteLn('Threads: ', ThreadCount, ', time: ', MilliSecondsBetween(Now, Start), ' ms');
end.
```

Вивід програми:

```
Primes: 348513, sum: 838596693108
Threads: 12, time: 201 ms
```

docs/concurrency/examples/primes_threading.pp

```
program primes_threading;
// Варіант 2: System.Threading (TParallel.For, TTask.Future).
// Кількість і сума простих чисел до N.
{$mode objfpc}{$H+}
{$modeswitch functionreferences}
{$modeswitch anonymousfunctions}
uses
  {$IFDEF UNIX}cthreads,{$ENDIF}
  SysUtils, DateUtils, System.Threading;

const
  N = 5000000;

function IsPrime(X: Integer): Boolean;
var
  D: Integer;
begin
  if X < 2 then Exit(False);
  if X mod 2 = 0 then Exit(X = 2);
  D := 3;
  while D * D <= X do
  begin
    if X mod D = 0 then Exit(False);
    Inc(D, 2);
  end;
  Result := True;
end;

var
  Count: Integer;
  Sum: Int64;
  Largest: specialize IFuture<Integer>;
  Start: TDateTime;
begin
  Start := Now;
  // Незалежна задача паралельно з циклом: найбільше просте до N.
  Largest := TTask.specialize Future<Integer>(
    function: Integer
    begin
      Result := N;
```

```
      while not IsPrime(Result) do
        Dec(Result);
    end);

  Count := 0;
  Sum := 0;
  // Пул сам ділить роботу між потоками й вирівнює навантаження.
  TParallel.&For(1, N,
    procedure(I: Integer)
    begin
      if IsPrime(I) then
      begin
        AtomicIncrement(Count);
        InterlockedExchangeAdd64(Sum, I);
      end;
    end);

  WriteLn('Primes: ', Count, ', sum: ', Sum);
  WriteLn('Largest prime <= N: ', Largest.Value);   // Value чекає результат
  WriteLn('Time: ', MilliSecondsBetween(Now, Start), ' ms');
end.
```

Вивід програми:

```
Primes: 348513, sum: 838596693108
Largest prime <= N: 4999999
Time: 122 ms
```

docs/concurrency/examples/report_classic.pp

```
program report_classic;
// Варіант 1: лише TThread з RTL (без System.Threading).
// Звіт збирається з 4 незалежних джерел різної тривалості; одне падає.
// Вимоги: виконати паралельно, загальний ліміт 2 с, показати помилки.
{$mode objfpc}{$H+}
uses
  {$IFDEF UNIX}cthreads,{$ENDIF}
  SysUtils, Classes, SyncObjs, DateUtils;

var
  Remaining: Integer;       // скільки задач ще не завершилось
  AllDone: TSimpleEvent;    // подія «усі завершились» — для таймауту

type
  // Базовий клас: лічильник і подія завершення — вручну.
  TJob = class(TThread)
  protected
    procedure Work; virtual; abstract;
    procedure Execute; override;
  end;

  TOrdersJob = class(TJob)
  public Orders: Int64;
  protected procedure Work; override;
  end;

  TTopCustomerJob = class(TJob)
  public Name: string;
  protected procedure Work; override;
  end;

  TAverageJob = class(TJob)
  public Average: Double;
  protected procedure Work; override;
  end;

  TRatesJob = class(TJob)
  public Rate: Double;
  protected procedure Work; override;
  end;

procedure TJob.Execute;
```

```
begin
  try
    Work;                   // виняток потрапить у FatalException
  finally
    if InterlockedDecrement(Remaining) = 0 then
      AllDone.SetEvent;
  end;
end;

procedure TOrdersJob.Work;
begin
  Sleep(400); Orders := 1234;
end;

procedure TTopCustomerJob.Work;
begin
  Sleep(250); Name := 'Acme Ltd';
end;

procedure TAverageJob.Work;
begin
  Sleep(300); Average := 87.5;
end;

procedure TRatesJob.Work;
begin
  Sleep(150);
  raise Exception.Create('Rates service: connection refused');
end;

var
  Jobs: array[0..3] of TJob;
  I: Integer;
  Start: TDateTime;
begin
  Start := Now;
  AllDone := TSimpleEvent.Create;
  Remaining := Length(Jobs);
  Jobs[0] := TOrdersJob.Create(False);
  Jobs[1] := TTopCustomerJob.Create(False);
  Jobs[2] := TAverageJob.Create(False);
  Jobs[3] := TRatesJob.Create(False);
  try
    // TThread.WaitFor не має таймауту, тому — власна подія.
    if AllDone.WaitFor(2000) <> wrSignaled then
      WriteLn('Timeout: report is incomplete');
    for I := 0 to High(Jobs) do
      Jobs[I].WaitFor;
    for I := 0 to High(Jobs) do       // помилки — перевіряти кожен потік
      if Assigned(Jobs[I].FatalException) then
        WriteLn('Error: ', Exception(Jobs[I].FatalException).Message);
    WriteLn('Orders: ', TOrdersJob(Jobs[0]).Orders);
    WriteLn('Top customer: ', TTopCustomerJob(Jobs[1]).Name);
    WriteLn('Average check: ', TAverageJob(Jobs[2]).Average:0:1);
    if not Assigned(Jobs[3].FatalException) then
      WriteLn('Rate: ', TRatesJob(Jobs[3]).Rate:0:2);
  finally
    for I := 0 to High(Jobs) do
      Jobs[I].Free;
    AllDone.Free;
  end;
  WriteLn('Time: ', MilliSecondsBetween(Now, Start), ' ms');
end.
```

Вивід програми:

```
Error: Rates service: connection refused
Orders: 1234
Top customer: Acme Ltd
Average check: 87.5
Time: 401 ms
```

docs/concurrency/examples/report_threading.pp

```
program report_threading;
// Варіант 2: System.Threading (IFuture, WaitForAll, EAggregateException).
// Звіт збирається з 4 незалежних джерел різної тривалості; одне падає.
// Вимоги: виконати паралельно, загальний ліміт 2 с, показати помилки.
{$mode objfpc}{$H+}
{$modeswitch functionreferences}
{$modeswitch anonymousfunctions}
uses
  {$IFDEF UNIX}cthreads,{$ENDIF}
  SysUtils, DateUtils, System.Threading;

var
  Orders: specialize IFuture<Int64>;
  TopCustomer: specialize IFuture<string>;
  Average, Rate: specialize IFuture<Double>;
  E: Exception;
  Start: TDateTime;
begin
  Start := Now;
  Orders := TTask.specialize Future<Int64>(
    function: Int64 begin Sleep(400); Result := 1234; end);
  TopCustomer := TTask.specialize Future<string>(
    function: string begin Sleep(250); Result := 'Acme Ltd'; end);
  Average := TTask.specialize Future<Double>(
    function: Double begin Sleep(300); Result := 87.5; end);
  Rate := TTask.specialize Future<Double>(
    function: Double
    begin
      Sleep(150);
      raise Exception.Create('Rates service: connection refused');
    end);

  try
    if not TTask.WaitForAll([Orders, TopCustomer, Average, Rate], 2000) then
      WriteLn('Timeout: report is incomplete');
  except
    on Agg: EAggregateException do    // усі помилки зібрано в одному місці
      for E in Agg do
        WriteLn('Error: ', E.Message);
  end;
  WriteLn('Orders: ', Orders.Value);
  WriteLn('Top customer: ', TopCustomer.Value);
  WriteLn('Average check: ', Average.Value:0:1);
  if Rate.Status = TTaskStatus.Completed then
    WriteLn('Rate: ', Rate.Value:0:2);
  WriteLn('Time: ', MilliSecondsBetween(Now, Start), ' ms');
end.
```

Вивід програми:

```
Error: Rates service: connection refused
Orders: 1234
Top customer: Acme Ltd
Average check: 87.5
Time: 401 ms
```

docs/concurrency/examples/batch_processing.pp

```
program batch_processing;
// Обробка порцій задач: 20 раундів по 500 незалежних задач (як сервер, що
// обробляє запити пачками). П'ять підходів на однаковій роботі; програма
// перевіряє, що всі дають той самий результат, і друкує таблицю часу.
{$mode objfpc}{$H+}
{$modeswitch functionreferences}
{$modeswitch anonymousfunctions}
uses
  {$IFDEF UNIX}cthreads,{$ENDIF}
  SysUtils, Classes, SyncObjs, System.Diagnostics, System.Threading;

const
```

```
  Rounds = 20;
  Jobs = 500;

var
  Results: array[0..Jobs - 1] of Int64;

// Одна задача: ~0,1 мс обчислень різної тривалості.
function Work(I: Integer): Int64;
var
  K: Integer;
begin
  Result := 0;
  for K := 1 to 20000 + (I mod 7) * 5000 do
    Result := Result + (K xor I) mod 7;
end;

function Checksum: Int64;
var
  I: Integer;
begin
  Result := 0;
  for I := 0 to Jobs - 1 do
    Result := Result + Results[I] * (I + 1);
end;

{ --- 1. Послідовно ------------------------------------------------------- }

procedure RunSequential;
var
  R, I: Integer;
begin
  for R := 1 to Rounds do
    for I := 0 to Jobs - 1 do
      Results[I] := Work(I);
end;

{ --- 2. Класика: потік на кожну задачу ----------------------------------- }

type
  TJobThread = class(TThread)
  private
    FIndex: Integer;
  protected
    procedure Execute; override;
  public
    constructor Create(AIndex: Integer);
  end;

constructor TJobThread.Create(AIndex: Integer);
begin
  FIndex := AIndex;
  inherited Create(False);
end;

procedure TJobThread.Execute;
begin
  Results[FIndex] := Work(FIndex);
end;

procedure RunThreadPerJob;
var
  R, I: Integer;
  T: array[0..Jobs - 1] of TJobThread;
begin
  for R := 1 to Rounds do
  begin
    for I := 0 to Jobs - 1 do
      T[I] := TJobThread.Create(I);
    for I := 0 to Jobs - 1 do
    begin
      T[I].WaitFor;
      T[I].Free;
```

```
    end;
  end;
end;

{ --- 3. Класика: робочі потоки + TThread.WaitFor ------------------------- }
{ --- 4. Класика «експерт»: робочі потоки + власна подія ----------------- }

var
  NextJob, Running: Integer;
  AllDone: TSimpleEvent;

type
  TWorker = class(TThread)
  protected
    procedure Execute; override;
  end;

procedure TWorker.Execute;
var
  I: Integer;
begin
  I := InterlockedIncrement(NextJob) - 1;       // кожен бере наступну задачу
  while I < Jobs do
  begin
    Results[I] := Work(I);
    I := InterlockedIncrement(NextJob) - 1;
  end;
  if InterlockedDecrement(Running) = 0 then
    AllDone.SetEvent;
end;

procedure RunWorkers(UseEvent: Boolean);
var
  R, I: Integer;
  W: array of TWorker;
begin
  SetLength(W, TThread.ProcessorCount);
  for R := 1 to Rounds do
  begin
    NextJob := 0;
    Running := Length(W);
    AllDone.ResetEvent;
    for I := 0 to High(W) do
      W[I] := TWorker.Create(False);
    if UseEvent then
      AllDone.WaitFor(INFINITE);   // прокидаємося одразу після останньої задачі
    for I := 0 to High(W) do
    begin
      W[I].WaitFor;                // у головному потоці опитує раз на 100 мс
      W[I].Free;
    end;
  end;
end;

{ --- 5. System.Threading ------------------------------------------------- }

procedure RunParallelFor;
var
  R: Integer;
begin
  for R := 1 to Rounds do
    TParallel.&For(0, Jobs - 1,
      procedure(I: Integer)
      begin
        Results[I] := Work(I);
      end);
end;

{ --- Заміри -------------------------------------------------------------- }

type
  TRunner = procedure;
```

```
var
  Reference: Int64;
  BaseMs: Int64;

procedure Measure(const Name: string; Run: TRunner);
var
  W: TStopWatch;
  Ms: Int64;
begin
  FillChar(Results, SizeOf(Results), 0);
  W := TStopWatch.StartNew;
  Run();
  Ms := W.ElapsedMilliseconds;
  if BaseMs = 0 then
  begin
    BaseMs := Ms;
    Reference := Checksum;
  end;
  WriteLn(Format('%-40s %6d ms  x%5.2f  %s',
    [Name, Ms, BaseMs / Ms, BoolToStr(Checksum = Reference, 'OK', 'WRONG RESULT')]));
end;

procedure RunWorkersWaitFor; begin RunWorkers(False); end;
procedure RunWorkersEvent;   begin RunWorkers(True);  end;

begin
  AllDone := TSimpleEvent.Create;
  WriteLn(Format('%d rounds x %d jobs, %d logical CPUs', [Rounds, Jobs, TThread.ProcessorCount]));
  Measure('1. Sequential', @RunSequential);
  Measure('2. TThread per job', @RunThreadPerJob);
  Measure('3. Worker threads + WaitFor', @RunWorkersWaitFor);
  Measure('4. Worker threads + event (expert)', @RunWorkersEvent);
  Measure('5. TParallel.For', @RunParallelFor);
  AllDone.Free;
end.
```

Вивід програми:

```
20 rounds x 500 jobs, 12 logical CPUs
1. Sequential                               926 ms  x 1.00  OK
2. TThread per job                          898 ms  x 1.03  OK
3. Worker threads + WaitFor                2031 ms  x 0.46  OK
4. Worker threads + event (expert)          190 ms  x 4.87  OK
5. TParallel.For                            160 ms  x 5.79  OK
```

docs/concurrency/examples/showcase_pipeline.pp

```
program showcase_pipeline;
// Порівняння на 4 прикладних задачах:
//   Стандартний підхід (класи TThread + ручна синхронізація)
//   проти System.Threading (TParallel.For + Break/Stop/Exceptions, WaitForAny, Join).
{$mode objfpc}{$H+}
{$modeswitch functionreferences}
{$modeswitch anonymousfunctions}
uses
  {$IFDEF UNIX}cthreads,{$ENDIF}
  SysUtils, Classes, SyncObjs, System.Diagnostics, System.Threading;

const
  N = 200000;

function ExpensiveVerify(Index: Integer): Int64;
var
  K: Integer;
  V: UInt64;
begin
  V := UInt64(Index) * 2654435761 + 17;
  for K := 1 to 600 do
    V := (V xor (V shr 13)) * 1274126177 + UInt64(K);
  Result := Int64(V and $7FFFFFFF);
end;
```

```
function IsCorrupted(Index: Integer): Boolean; inline;
begin
  Result := (Index = 24000) or (Index = 65000) or (Index = 150000);
end;

{ ============================================================================
  1. Пошук першого пошкодженого запису за індексом (Break)
  ============================================================================ }

var
  NextIdx1, LowestIdx1: Integer;

type
  TBreakWorker = class(TThread)
  protected
    procedure Execute; override;
  end;

procedure TBreakWorker.Execute;
var
  I, CurMin, Old: Integer;
begin
  while True do
  begin
    I := InterlockedIncrement(NextIdx1) - 1;
    CurMin := InterlockedCompareExchange(LowestIdx1, 0, 0);
    if (I >= N) or (I >= CurMin) then Exit;
    ExpensiveVerify(I);
    if IsCorrupted(I) then
    begin
      // Атомарно оновлюємо найменший знайдений індекс (CAS-цикл вручну)
      repeat
        CurMin := LowestIdx1;
        if I >= CurMin then Break;
        Old := InterlockedCompareExchange(LowestIdx1, I, CurMin);
      until Old = CurMin;
      Exit;
    end;
  end;
end;

procedure Stage1_Break;
var
  W: array of TBreakWorker;
  I: Integer;
  SW: TStopWatch;
  ClassicMs, ThreadingMs: Int64;
  Res: TParallel.TLoopResult;
begin
  // А) Стандартний TThread: клас потоку + ручний CAS-цикл мінімуму + WaitFor
  SW := TStopWatch.StartNew;
  NextIdx1 := 0;
  LowestIdx1 := MaxInt;
  SetLength(W, TThread.ProcessorCount);
  for I := 0 to High(W) do W[I] := TBreakWorker.Create(False);
  for I := 0 to High(W) do begin W[I].WaitFor; W[I].Free; end;
  ClassicMs := SW.ElapsedMilliseconds;

  // Б) System.Threading: State.Break сам відстежує LowestBreakIteration
  SW := TStopWatch.StartNew;
  Res := TParallel.&For(0, N - 1,
    procedure(Idx: Integer; State: TParallel.TLoopState)
    begin
      if State.ShouldExit then Exit;
      ExpensiveVerify(Idx);
      if IsCorrupted(Idx) then
        State.Break;
    end);
  ThreadingMs := SW.ElapsedMilliseconds;

  WriteLn(Format('1. Ordered Break (index=%d=%d):   TThread %4d ms (38 lines) | System.Threading %3d ms (10 line
```

```
    [LowestIdx1, Integer(Res.LowestBreakIteration), ClassicMs, ThreadingMs]));
end;

{ ============================================================================
  2. Пошук будь-якого придатного елемента з миттєвою зупинкою (Stop)
  ============================================================================ }

var
  NextIdx2, StopFlag2, FoundIdx2: Integer;

type
  TStopWorker = class(TThread)
  protected
    procedure Execute; override;
  end;

procedure TStopWorker.Execute;
var
  I: Integer;
begin
  while InterlockedCompareExchange(StopFlag2, 0, 0) = 0 do
  begin
    I := InterlockedIncrement(NextIdx2) - 1;
    if I >= N then Exit;
    if (ExpensiveVerify(I) mod 9973 = 0) and (I > 1000) then
    begin
      InterlockedCompareExchange(FoundIdx2, I, -1);
      InterlockedExchange(StopFlag2, 1);
      Exit;
    end;
  end;
end;

procedure Stage2_Stop;
var
  W: array of TStopWorker;
  I, FoundPar: Integer;
  SW: TStopWatch;
  ClassicMs, ThreadingMs: Int64;
  Res: TParallel.TLoopResult;
begin
  // А) Стандартний TThread: клас потоку + спільний прапорець зупинки
  SW := TStopWatch.StartNew;
  NextIdx2 := 0;
  StopFlag2 := 0;
  FoundIdx2 := -1;
  SetLength(W, TThread.ProcessorCount);
  for I := 0 to High(W) do W[I] := TStopWorker.Create(False);
  for I := 0 to High(W) do begin W[I].WaitFor; W[I].Free; end;
  ClassicMs := SW.ElapsedMilliseconds;

  // Б) System.Threading: State.Stop
  FoundPar := -1;
  SW := TStopWatch.StartNew;
  Res := TParallel.&For(0, N - 1,
    procedure(Idx: Integer; State: TParallel.TLoopState)
    begin
      if State.ShouldExit then Exit;
      if (ExpensiveVerify(Idx) mod 9973 = 0) and (Idx > 1000) then
      begin
        InterlockedCompareExchange(FoundPar, Idx, -1);
        State.Stop;
      end;
    end);
  ThreadingMs := SW.ElapsedMilliseconds;

  WriteLn(Format('2. Immediate Stop (completed=%s): TThread %4d ms (30 lines) | System.Threading %3d ms (11 line
    [BoolToStr(Res.Completed, 'T', 'F'), ClassicMs, ThreadingMs]));
end;

{ ============================================================================
  3. Пакетна валідація з перехопленням винятків
```

```
  ============================================================================ }

type
  TValidateWorker = class(TThread)
  private
    FLow, FHigh: Integer;
  protected
    procedure Execute; override;
  public
    constructor Create(ALow, AHigh: Integer);
  end;

constructor TValidateWorker.Create(ALow, AHigh: Integer);
begin
  FLow := ALow;
  FHigh := AHigh;
  inherited Create(False);
end;

procedure TValidateWorker.Execute;
var
  I: Integer;
begin
  for I := FLow to FHigh do
  begin
    ExpensiveVerify(I);
    if I = 150 then
      raise Exception.Create('Invalid checksum at record #150');
  end;
end;

procedure Stage3_Exceptions;
var
  W: array[0..3] of TValidateWorker;
  I: Integer;
  ClassicMsg, ThreadingMsg: string;
  SW: TStopWatch;
  ClassicMs, ThreadingMs: Int64;
  E: Exception;
begin
  // А) Стандартний TThread: виняток осідає у FatalException кожного потоку;
  //    якщо не перевірити вручну перед Free — помилку буде тихо втрачено.
  SW := TStopWatch.StartNew;
  ClassicMsg := '';
  for I := 0 to High(W) do
    W[I] := TValidateWorker.Create(I * 250, (I + 1) * 250 - 1);
  for I := 0 to High(W) do
  begin
    W[I].WaitFor;
    if Assigned(W[I].FatalException) then
      ClassicMsg := Exception(W[I].FatalException).Message;
    W[I].Free;
  end;
  ClassicMs := SW.ElapsedMilliseconds;

  // Б) System.Threading: виняток автоматично прокидається як EAggregateException
  SW := TStopWatch.StartNew;
  ThreadingMsg := '';
  try
    TParallel.&For(0, 999,
      procedure(Idx: Integer)
      begin
        ExpensiveVerify(Idx);
        if Idx = 150 then
          raise Exception.Create('Invalid checksum at record #150');
      end);
  except
    on Agg: EAggregateException do
      for E in Agg do
        ThreadingMsg := E.Message;
  end;
  ThreadingMs := SW.ElapsedMilliseconds;
```

```
  WriteLn(Format('3. Exception handling (match=%s):  TThread %4d ms (36 lines) | System.Threading %3d ms (14 lin
    [BoolToStr((ClassicMsg <> '') and (ClassicMsg = ThreadingMsg), 'TRUE', 'FALSE'),
     ClassicMs, ThreadingMs]));
end;

{ ============================================================================
  4. Найшвидша репліка (WaitForAny) + паралельне зведення (Join)
  ============================================================================ }

var
  FastestEvent: TSimpleEvent;
  WinnerIdx: Integer;

type
  TReplicaThread = class(TThread)
  private
    FIdx, FDelayMs: Integer;
  public
    Response: string;
    constructor Create(AIdx, ADelayMs: Integer; const AName: string);
  protected
    procedure Execute; override;
  end;

  TSumThread = class(TThread)
  private
    FParity: Integer;
  public
    Total: Int64;
    constructor Create(AParity: Integer);
  protected
    procedure Execute; override;
  end;

constructor TReplicaThread.Create(AIdx, ADelayMs: Integer; const AName: string);
begin
  FIdx := AIdx;
  FDelayMs := ADelayMs;
  Response := AName;
  inherited Create(False);
end;

procedure TReplicaThread.Execute;
begin
  Sleep(FDelayMs);
  if InterlockedCompareExchange(WinnerIdx, FIdx, -1) = -1 then
    FastestEvent.SetEvent;
end;

constructor TSumThread.Create(AParity: Integer);
begin
  FParity := AParity;
  inherited Create(False);
end;

procedure TSumThread.Execute;
var
  I: Integer;
begin
  Total := 0;
  for I := 0 to 49999 do
    if I mod 2 = FParity then
      Inc(Total, I);
end;

procedure Stage4_WaitAnyAndJoin;
var
  Rep: array[0..2] of TReplicaThread;
  S0, S1: TSumThread;
  ClassicFastest: string;
  ClassicTotal, SumEven, SumOdd: Int64;
```

```
  R0, R1, R2: specialize IFuture<string>;
  Replicas: array[0..2] of specialize IFuture<string>;
  Fastest, I: Integer;
  SW: TStopWatch;
  ClassicMs, ThreadingMs: Int64;
begin
  // А) Стандартний TThread: TThread.WaitFor не вміє чекати "будь-який" потік,
  //    тому потрібні TSimpleEvent, атомарний WinnerIdx і 2 класи потоків.
  SW := TStopWatch.StartNew;
  FastestEvent := TSimpleEvent.Create;
  WinnerIdx := -1;
  Rep[0] := TReplicaThread.Create(0, 250, 'eu-west');
  Rep[1] := TReplicaThread.Create(1, 40,  'eu-central');
  Rep[2] := TReplicaThread.Create(2, 180, 'us-east');
  FastestEvent.WaitFor(1000);
  ClassicFastest := Rep[WinnerIdx].Response;

  S0 := TSumThread.Create(0);
  S1 := TSumThread.Create(1);
  S0.WaitFor; S1.WaitFor;
  ClassicTotal := S0.Total + S1.Total;
  S0.Free; S1.Free;
  ClassicMs := SW.ElapsedMilliseconds;
  for I := 0 to 2 do begin Rep[I].WaitFor; Rep[I].Free; end;
  FastestEvent.Free;

  // Б) System.Threading: IFuture<string> + WaitForAny + TParallel.Join
  SW := TStopWatch.StartNew;
  R0 := TTask.specialize Future<string>(function: string begin Sleep(250); Result := 'eu-west'; end);
  R1 := TTask.specialize Future<string>(function: string begin Sleep(40);  Result := 'eu-central'; end);
  R2 := TTask.specialize Future<string>(function: string begin Sleep(180); Result := 'us-east'; end);
  Replicas[0] := R0; Replicas[1] := R1; Replicas[2] := R2;
  Fastest := TTask.WaitForAny([R0, R1, R2], 1000);

  SumEven := 0; SumOdd := 0;
  TParallel.Join([
    procedure var K: Integer; begin for K := 0 to 49999 do if K mod 2 = 0 then Inc(SumEven, K); end,
    procedure var K: Integer; begin for K := 0 to 49999 do if K mod 2 = 1 then Inc(SumOdd, K); end
  ]).Wait;
  ThreadingMs := SW.ElapsedMilliseconds;
  TTask.WaitForAll([R0, R1, R2]);

  WriteLn(Format('4. WaitForAny + Join (%s, %d): TThread %4d ms (64 lines) | System.Threading %3d ms (16 lines)'
    [Replicas[Fastest].Value, SumEven + SumOdd, ClassicMs, ThreadingMs]));
  Assert((ClassicFastest = Replicas[Fastest].Value) and (ClassicTotal = SumEven + SumOdd));
end;

begin
  WriteLn('=== Standard TThread vs System.Threading (', TThread.ProcessorCount, ' logical CPUs) ===');
  Stage1_Break;
  Stage2_Stop;
  Stage3_Exceptions;
  Stage4_WaitAnyAndJoin;
  WriteLn('Total helper thread classes:        TThread = 5 classes (168 lines) | System.Threading = 0 classes (5
end.
```

Вивід програми:

```
=== Standard TThread vs System.Threading (12 logical CPUs) ===
1. Ordered Break (index=24000=24000):   TThread  101 ms (38 lines) | System.Threading  16 ms (10 lines)
2. Immediate Stop (completed=F): TThread  101 ms (30 lines) | System.Threading   6 ms (11 lines)
3. Exception handling (match=TRUE):  TThread  100 ms (36 lines) | System.Threading   1 ms (14 lines)
4. WaitForAny + Join (eu-central, 1249975000): TThread  141 ms (64 lines) | System.Threading  41 ms (16 lines)
Total helper thread classes:        TThread = 5 classes (168 lines) | System.Threading = 0 classes (51 lines)
```